实验二：搜索算法实验报告

GitHub 仓库：https://github.com/shixuhui-ops/AI-Search-Experiment-2

实验二：搜索算法实验报告

一、实验目标

本实验实现并测试了 A* 搜索、加权地形 A* 搜索、Minimax 和 Alpha-Beta 剪枝算法，并将 A* 与博弈搜索结合用于竞争环境下的宝藏分配问题。

二、算法设计

1. 迷宫 A*

状态为网格中的坐标 `(row, col)`。

□ `g(n)` 表示从起点到当前节点的实际移动代价。

□ `h(n)` 使用当前节点到终点的曼哈顿距离。

□ `f(n) = g(n) + h(n)`。

□ 每次从优先队列中取出 `f(n)` 最小的节点。

□ 障碍物 `X` 不允许进入。

□ 每次移动的代价为 1。

2. 加权地形 A*

加权地图中，进入相邻格子的代价等于目标格子的地形值，进入终点 `E` 的代价为 0。

使用曼哈顿距离乘以地图中的最小正地形代价作为启发函数。该启发函数不会高估真实剩余代价，因此具有可采纳性。

3. Minimax

根节点为 MAX 玩家，下一层为 MIN 玩家，双方交替进行。

□ MAX 节点选择子节点中的最大值。

□ MIN 节点选择子节点中的最小值。

□ 叶子节点直接返回对应分数。

4. Alpha-Beta 剪枝

Alpha-Beta 剪枝在 Minimax 基础上维护两个边界：

□ `alpha`：MAX 当前已经找到的最好值。

□ `beta`：MIN 当前已经找到的最好值。

当 `alpha >= beta` 时，后续分支不可能影响最终决策，因此直接剪枝。

三、实验结果

1. 迷宫 A*

最短路径为：

[(0, 0), (0, 1), (0, 2), (1, 2),

(1, 3), (1, 4), (2, 4), (3, 4)]

路径共移动 7 步。

2. 加权地形 A*

最小总移动代价为：

7

算法选择了总代价更低的路径，而不是简单地选择经过格子数量最少的路径。

3. Minimax 与 Alpha-Beta

Minimax 结果：

5

Alpha-Beta 结果：

5

两种算法结果一致，说明 Alpha-Beta 剪枝不会改变 Minimax 的最终结果。

4. 宝藏距离矩阵

点的顺序为：

[A, B, T1, T2, T3, T4]

距离矩阵为：

[0, 12, 10, 4, 8, 8]

[12, 0, 8, 8, 4, 10]

[10, 8, 0, 6, 6, 12]

[4, 8, 6, 0, 6, 6]

[8, 4, 6, 6, 0, 6]

[8, 10, 12, 6, 6, 0]

5. 宝藏博弈

宝藏价值设置为：

T1 = 9

T2 = 7

T3 = 8

T4 = 7

Minimax 在搜索深度 H=2 时的结果：

最优首步：T2

最优收益差：-1

Minimax 在搜索深度 H=4 时的结果：

最优首步：T3

最优收益差：3

Alpha-Beta 在 H=2 和 H=4 时得到相同结果：

H=2：T2，-1

H=4：T3，3

四、结果分析

在迷宫问题中，曼哈顿距离适合四方向移动，并且不会高估真实剩余代价，因此 A* 可以找到最短路径。

在加权地形问题中，路径长度不再是唯一标准。算法必须综合考虑每个地形的进入代价，因此可能选择更长但总代价更低的路径。

Alpha-Beta 剪枝通过维护 alpha 和 beta 边界，跳过不可能影响最终结果的分支。它与 Minimax 得到相同结果，但可以减少节点访问数量。

在宝藏分配问题中，搜索深度会影响最终决策。H=2 只考虑较短的后续过程，而 H=4 能够考虑完整的四轮博弈，因此两个深度下的最优首步不同。

五、实验总结

本实验完成了迷宫 A*、加权地形 A*、Minimax、Alpha-Beta 剪枝以及综合宝藏分配问题的实现。

所有测试结果均与题目给出的预期结果一致。实验表明，启发函数可以提高 A* 的搜索效率，Alpha-Beta 可以减少 Minimax 的无效搜索，而博弈搜索深度会直接影响最终决策。

六、性能优化对比

在相同迷宫上分别使用 h=0 和曼哈顿距离启发函数进行测试。

h=0 时扩展节点数为 16 个；使用曼哈顿距离时扩展节点数为 14 个。

节点数减少了 2 个，减少比例为 12.5%。这说明在保证最优路径的前提下，合理的启发函数能够减少搜索范围，提高 A* 的搜索效率。

七、Minimax 与 Alpha-Beta 性能对比

在相同博弈树上，Minimax 共访问 15 个节点，Alpha-Beta 共访问 14 个节点。

Alpha-Beta 剪枝减少访问 1 个节点，最终结果仍然保持为 5。

这说明 Alpha-Beta 不改变最优决策，只通过 alpha 和 beta
上界排除不可能影响结果的分支。实际搜索树规模增大时，剪枝收益会更加明显。

八、边界情况测试

当迷宫中的起点和终点之间不存在可行路径时，A* 返回空列表 []。

当加权地图中的起点和终点之间不存在可行路径时，加权 A* 返回 -1。

测试结果表明，程序能够正确处理无解输入，不会进入死循环，也不会返回错误路径。

九、Alpha-Beta 访问顺序分析

在当前测试树中，保持子节点原顺序和反转子节点顺序时，Alpha-Beta 的结果均为 5，访问节点数均为 14。

本测试中访问顺序没有造成节点数差异，说明该树规模较小，且当前分支界限不足以产生额外剪枝。在更深、更宽的博弈树中，优先访问更可能产生极值的节点通常能够获得更高的剪枝收益。

十、自动化测试

使用 Python unittest 对迷宫 A*、无解判断、加权地形 A*、Minimax、Alpha-Beta 和宝藏博弈进行了自动化测试。

共执行 5 个测试用例，全部通过，结果为 OK。这说明核心算法能够稳定满足题目要求，并且边界情况得到正确处理。

十一、运行时间基准测试

在相同测试环境下，A* 使用曼哈顿距离后扩展节点数从 16 个减少到 14 个，运行时间也由约 0.051 ms 降低到约 0.032 ms。

Minimax 访问 11 个节点，Alpha-Beta 访问 9 个节点。由于测试树规模较小，Alpha-Beta
的运行时间可能受到函数调用和计时误差影响，但节点访问数量已经减少，说明剪枝逻辑有效。

十二、大规模博弈树测试

在深度为 4、每个节点有 3 个子节点的博弈树上，原始访问顺序访问 73 个节点，反转访问顺序访问 45 个节点。

两种访问顺序得到的最优值均为 141，但反转顺序少访问 28 个节点，说明 Alpha-Beta
的剪枝效率与子节点访问顺序密切相关。合理的节点排序可以显著提高剪枝效果。

十三、记忆化搜索优化

宝藏 Minimax 中，相同的玩家位置、剩余宝藏、累计得分和当前回合可能通过不同路径重复出现。

程序使用 lru_cache 对搜索状态进行缓存，避免重复计算相同状态，从而降低重复搜索开销。缓存只应用于 Minimax，因为普通 Minimax 的状态值是确定的；Alpha-Beta 的返回结果可能受到 alpha 和 beta 边界影响，因此没有直接缓存剪枝结果。

十四、测试覆盖率

使用 coverage 对自动化测试进行统计。

测试结果为：solution.py 覆盖率 76%，整体项目覆盖率 79%。

核心算法的主要路径已经覆盖，剩余未覆盖部分主要是演示入口、异常分支和少量边界代码。

十五、扩展测试结果

在原有测试基础上，增加了加权地图无解测试、树反转测试和 Alpha-Beta 访问顺序测试。

自动化测试数量由 6 个增加到 9 个，全部通过。

使用 coverage 统计后，solution.py 覆盖率达到 87%，项目整体覆盖率达到 88%。

十六、核心实验结果对比表

| 实验项目 | 方法或参数 | 结果 | 性能数据 |

|---|---|---|---|

| 迷宫寻路 | A*，h=0 | 路径长度 8 | 扩展 16 个节点 |

| 迷宫寻路 | A*，曼哈顿启发式 | 路径长度 8 | 扩展 14 个节点 |

| 加权地形 | 加权 A* | 最小代价 7 | 正确避开高代价地形 |

| 博弈树 | Minimax | 最优值 5 | 访问 11 个节点 |

| 博弈树 | Alpha-Beta | 最优值 5 | 访问 9 个节点 |

| 宝藏博弈 | H=2 | T2，收益差 -1 | Minimax 与 Alpha-Beta 一致 |

| 宝藏博弈 | H=4 | T3，收益差 3 | Minimax 与 Alpha-Beta 一致 |

表中可以看出，启发函数减少了 A* 的节点扩展，Alpha-Beta 减少了博弈树搜索节点，同时没有改变最终决策结果。

十七、Alpha-Beta 搜索过程示意图

MAX

/ \

MIN MIN

/ \ / \

MAX MAX MAX MAX

/ \ / \ / \ / \

3 5 6 9 1 2 4 7

搜索从根节点开始，MAX 层选择较大值，MIN 层选择较小值。当当前 alpha 值不小于 beta 值时，后续分支不再可能改变祖先节点的决策，因此可以剪枝。